KB: A Facts-First Codebase Knowledge System with Multi-Objective Exploration Loss Evaluation

Jacob Rosenzweig

Fintary

jacob.rosenzweig@fintary.com

KB v1.2.0 • July 5, 2026

Abstract

LLM-based developer agents suffer from two compounding inefficiencies: each session begins with an empty context window, forcing costly re-exploration of the codebase, and current evaluation frameworks measure only whether the final answer is correct, hiding the exploration cost that produced it. We present two complementary contributions.

KB is a local-first, facts-first codebase knowledge system that converts knowledge maintenance into a side-effect of normal development work. KB indexes source code deterministically via AST analysis (tree-sitter) and documentation via sentence-level fact extraction, storing atomic facts in a SQLite graph with typed edges and optional embedding vectors computed at init. At query time, a multi-pond BFS loop merges lexical (BM25/FTS5), semantic (embedding), and graph-walk evidence streams into a ranked fact pool, exiting on a scored sufficiency condition. A judge-in-the-loop fact curator then hard-drops off-topic facts and refills reported gaps before one-shot synthesis returns plain prose.

MOEL (Multi-Objective Exploration Loss) is a quantitative evaluation framework that measures agent efficiency under three controlled conditions: Condition N (raw filesystem baseline), Condition K (KB-equipped), and Condition O (oracle context injection). MOEL combines three normalized loss terms—LLM jury semantic loss (L_{jury}), trajectory inefficiency ($L_{\text{trajectory}}$), and resource consumption (L_{resource})—into a single weighted scalar L_{MOEL} in a deep evaluation with toy-tool N/K/O conditions. Paired harvest evaluation instead compares `kb query` (K) to a real headless coding agent (N) via ΔS.

The headline metric is a budget-normalized success score $S = 0.60\, Q_{\text{adeq}} + 0.30\, E_{\text{tok}} + 0.10\, E_{\text{speed}}$ that blends adequacy-weighted answer quality (correctness, usefulness, and relevance with diminishing returns above τ=3) with token economy and latency, scored identically for KB-equipped and control runs. The project grade is $\Delta S = S_K - S_N$ from a single paired evaluation (Eq. 9). In the latest paired harvest evaluations (July 5, 2026), KB self-check yields $\Delta S = +0.166$ ($S_K = 0.869$, $S_N = 0.703$) and raylib yields $\Delta S = -0.057$ ($S_K = 0.673$, $S_N = 0.730$), with paired K-vs-N evaluations on both `kb` and `raylib` suites (July 5, 2026).

1 Introduction

Large language models (LLMs) have dramatically changed how developers interact with codebases, enabling agents to autonomously resolve real-world software engineering issues [11]. Yet the dominant interaction model remains *ephemeral*: each agent session begins with an empty context window, forcing the model to re-discover facts about the codebase from scratch. This creates a compounding inefficiency—teams that rely on LLM assistants must accept that every question triggers a fresh, expensive, and error-prone tour of the filesystem.

Two failure modes characterize this status quo. First, **exploration waste**: agents read far more source files than necessary before reaching an answer, because there is no prior accumulation of structured knowledge to query. On representative SWE-Bench tasks, text-based agents spend up to 21 steps navigating via grep and line-range reads to locate a single target function—work that structured pre-indexed facts resolve in two or three queries [5]. Second, **context dilution**: irrelevant context degrades LLM reasoning [8], and brittle string-matching edits fail when whitespace or formatting drifts [5].

Existing retrieval-augmented generation (RAG) systems [6] address the first problem partially, but are designed for static document corpora, not living codebases with interleaved code, documentation, and decision history. Code-search benchmarks such as CodeSearchNet [4] evaluate individual retrieval steps in isolation, ignoring the sequential cost of multi-step agent exploration.

We present **KB**, a local-first, *facts-first* codebase knowledge system that:

(1) indexes a repository's code and documentation into a persistent SQLite store of atomic, citable facts via deterministic AST analysis and sentence-level fact extraction;

(2) exposes those facts to LLM agents through a multi-pond BFS retrieval loop that merges lexical (BM25), semantic (embedding similarity on stored vectors), and graph-walk evidence streams, then a judge-in-the-loop *fact curator* that hard-drops off-topic hits before one-shot synthesis—without manual query engineering.

Critically, KB's value is only as compelling as the evidence that it actually reduces agent exploration cost. Rubric-based Q&A scoring—the standard in prior agent evaluation—captures answer quality but is blind to *how* the agent arrived at that answer. A system that reads 40 files before synthesizing a correct response and one that queries two facts look identical under a pass/fail quality signal.

To fill this gap, we introduce the **Multi-Objective Exploration Loss (MOEL)** evaluation framework. Paired harvest evaluation scores both KB (K) and a real-agent control (N) on the same question pack per suite and reports the headline grade $\Delta S = S_K - S_N$ (Eq. 9). A complementary deep evaluation runs tasks under three conditions (N, K, O) and aggregates exploration loss into L_{MOEL}.

Figure 1 illustrates the end-to-end system. Our contributions are:

1. **KB**: a facts-first codebase knowledge system with deterministic AST indexing, init-time fact embeddings for semantic scoring, a multi-pond hybrid retrieval orchestrator, and a post-retrieval relevance curator that filters the fact pool before synthesis.

2. **MOEL**: a multi-objective evaluation framework comprising three normalized loss functions (L_{jury}, $L_{\mathrm{trajectory}}$, L_{resource}), bias-mitigated LLM jury scoring, and logistic-regression calibration on a 20-sample human-annotated set.

3. **Empirical results**: paired harvest evaluation on fixed `kb` and `raylib` repository snapshots compares `kb query` (K) to a real coding agent (N) under identical scoring (Table 2). Latest headline grades: $\Delta S = +0.166$ (kb self-check) and $\Delta S = -0.057$ (raylib), updated July 5, 2026.

[FIGURE 1 — System Overview]

A three-panel horizontal flow diagram:
Left panel — Developer working directory with source files, markdown, and version history feeding into `kb init` (AST indexer + sentence segmenter + fact extractor).
Center panel — SQLite knowledge store: `facts` table (doc + code facts), `fact_edges` graph, `fact_embeddings` semantic index, `documents` (stored prose sources).
Right panel — LLM agent loop: `kb query` dispatches multi-pond BFS retrieval, a fact curator (relevance judge) trims the pool, passes kept facts to LLM synthesis, returns plain grounded prose.
Below all panels: paired harvest evaluation reporting $\Delta S = S_K - S_N$ (K vs. real agent); deep MOEL evaluation (N/K/O toy-tool conditions) reporting L_{MOEL}.

Figure 1: End-to-end KB system. Primary harvest grade: ΔS (K vs. real agent); deep MOEL evaluation adds L_{MOEL} under toy-tool N/K/O conditions.

2 Related Work

2.1 LLM-Based Code Agents and Tools

SWE-Agent [11] navigates repositories through iterative file-read and string-edit operations. Agentless [10] takes a non-agentic approach—a fixed localization-then-repair pipeline that deliberately avoids giving the LLM autonomous tool-use—yet still requires direct file reads without pre-indexed context. AutoCodeRover [12] augments localization with AST-based code search APIs (e.g., `search_method_in_file`) that retrieve class and method implementations from a structured AST index. CODESTRUCT [5] replaces text-based edits with AST-grounded primitives (`readCode`/`editCode`), reducing input tokens by 12–38% and improving SWE-Bench Verified Pass@1 by up to 5.0pp. KB is complementary to these systems: whereas CODESTRUCT improves the *action interface* once a target is located, KB improves *context retrieval* by replacing filesystem exploration with structured fact lookup. The two approaches can be composed.

2.2 Retrieval-Augmented Generation

RAG [6] grounds LLM answers in retrieved passages, but standard RAG pipelines are designed for static document collections. They do not model the inter-entity relationships that characterize codebases (function calls, class hierarchies, module imports), and they treat retrieval as a one-shot lookup rather than an adaptive loop. KB's multi-pond BFS loop merges five evidence streams per pass and exits on a scored sufficiency condition, making retrieval quality a function of graph density rather than fixed index size.

2.3 LLM Evaluation Frameworks

Evaluating LLM outputs with another LLM (LLM-as-judge) is established in MT-Bench [13], which identifies position bias, verbosity bias, and self-enhancement bias as the primary failure modes. AlpacaFarm [2] applies a related idea—using LLMs to simulate human preference feedback—for RLHF training at lower cost. PandaLM [9] trains a dedicated judge model for reliable instruction-tuning evaluation. MOEL's jury module addresses the same biases at inference time through position debiasing (swapped-order consistency), verbosity normalization ($\log_{1+}$ length scaling), self-enhancement down-weighting ($\times 0.5$ for same-family judges), and family diversity enforcement—without requiring a fine-tuned judge. Unlike prior work, MOEL is not outcome-only: deep MOEL evaluation combines the LLM jury with trajectory and resource losses so that efficiency failures are numerically visible; paired harvest evaluation adds ΔS against a real agent baseline.

2.4 Agent Efficiency Measurement

SWE-ContextBench measures token cost and cache efficiency across context injection strategies; CodeScaleBench evaluates tool validity across repository scale classes. Neither framework provides a unified scalar loss that spans correctness, trajectory, and resource consumption simultaneously. The closest prior work is SWE-Atlas, which verifies agent-declared file manifests against live repository diff output. MOEL's `ManifestValidator` adopts the same live-diff approach and extends it with a `MutationValidator` that applies tree-sitter AST body stubs to confirm causal linkage between agent changes and test coverage.

2.5 AST-Based Code Analysis

Tree-sitter and GumTree [3] provide fine-grained AST differencing and incremental parsing. code2vec [1] uses AST path embeddings for code representation learning. KB's code facts are derived deterministically from tree-sitter exports; MOEL's programmatic validators (manifest and mutation checks) extend SWE-Atlas-style live-diff verification with AST-grounded causal linkage.

3 KB: A Facts-First Codebase Knowledge System

3.1 Knowledge Representation

KB represents a codebase as a set of atomic, citable *facts* stored in a SQLite database. Each fact is a natural-language sentence paired with:

- a `source_kind` (`import_code` for AST-derived facts, `import_doc` for sentence-segmented markdown);
- a `source_ref` (e.g., `code:src/tools/indexer.ts@parseAST`);
- optional embedding vector for semantic search (computed at end of `kb init` when an embedder is available; see below);
- a soft-deletion tombstone for incremental invalidation.

Facts are connected by typed edges in a `fact_edges` table, forming a lightweight semantic graph. This graph enables breadth-first neighborhood expansion during retrieval—a capability not available in flat vector indices.

3.2 Indexing Pipeline

`kb init` executes a fully deterministic, two-phase indexing pipeline. No LLM is invoked and no API cost is incurred.

Phase 1 — AST Code Indexing. For each source file, KB invokes the `web-tree-sitter` WASM runtime with per-language export queries to extract named declarations (functions, classes, interfaces, type aliases, constants, module items). Each symbol becomes a code fact with its qualified source reference. Import edges between symbols are recorded in `fact_edges`, forming the structural backbone of the graph.

Phase 2 — Document Fact Extraction. Human-readable sources (README files, markdown documentation) are segmented into sentences via deterministic NLP segmentation. Each sentence becomes a document fact stored verbatim in the `facts` table.

Embeddings. After indexing completes, `kb init` runs a best-effort `embedAllFacts` pass: by default a local on-device sentence-transformer (`all-MiniLM-L6-v2`); set `KB_EMBEDDER=gemini` to opt into hosted Gemini embeddings. When no embedder is available, rows keep a legacy deterministic hash vector and semantic scoring degrades gracefully—init never blocks on embedding failure.

[FIGURE 2 — KB Indexing and Retrieval Architecture]

A two-column diagram.
Left column (Init): Source files → tree-sitter AST parser → named declarations → `facts` (code). Markdown/README → sentence segmenter → `facts` (doc). Both feed into `fact_edges` (graph) and `fact_embeddings` (semantic).
Right column (Query): Query string → intent router → multi-pond BFS loop (labeled with the five evidence streams: edge neighbors, primary FTS, pond FTS, concept frontier, concept rows) → scored fact pool (lexical overlap + stored embeddings on doc facts) → sufficiency check ($\geq$ 20 facts $\geq$ 0.50) → **fact curator** (relevance judge; top-100 candidate cap, hard-drop + gap refill when > 12 facts) → LLM synthesis (plain prose) → evidence header. Arrow between columns labeled "graph-augmented query expansion".

Figure 2: KB indexing (left) and multi-pond BFS retrieval with post-retrieval curation (right).

The raylib snapshot indexed in our latest harvest (14,761 entities, 12,792 edges; Table 2) completes init+scan in minutes on a laptop with no LLM indexing cost.

3.3 Retrieval: Multi-Pond BFS

At query time, KB executes a *deep* retrieval loop that maintains a set of *exploration ponds*—independent BFS frontiers seeded from different sub-queries—so that one dominant code neighborhood cannot monopolize the evidence budget.

Each iteration of the loop merges five candidate streams:

1. **Edge neighbors**: one BFS hop from the active pond's frontier via `fact_edges`.
2. **Primary lexical**: BM25 [7] FTS5 search for the original query.
3. **Pond lexical**: FTS5 search for the active pond's sub-query.
4. **Concept frontier**: facts sharing any concept token in the active concept set.
5. **Concept rows**: broader concept-union expansion.

Facts from the first pass are promoted as *anchors* and receive a +0.10 score boost in later passes, ensuring stable grounding. Document facts also receive a semantic similarity term (cosine against stored embedding vectors); code facts lean on graph proximity instead. The loop exits when any of the following conditions is satisfied: (a) $\geq$ 20 facts score $\geq$ 0.50 and concept coverage is sufficient (*answerable plateau*), (b) an in-loop LLM sufficiency judge confirms answerability, (c) all ponds are exhausted, or (d) the fact budget (500 facts) is reached.

Graph-augmented query expansion may optionally rewrite or widen the query string before the loop begins, based on entity relationships in the stored graph. The *pre-expansion* user question is preserved for answer synthesis so graph symbol names do not pollute scaffold or prompt text.

3.4 Post-Retrieval Curation

The deep BFS loop is deliberately *broad*: graph-augmented expansion and multi-pond frontiers grow *reachable* evidence islands that include facts adjacent to, but not directly about, the user's question. A separate *in-loop* sufficiency judge (called every few iterations once $\geq$ 5 facts score $\geq$ 0.50) decides when to *stop collecting*; it does not remove off-topic facts from the pool.

When the scored pool exceeds 12 facts, a **fact curator** runs *before* synthesis. It is judge-in-the-loop, not a static score floor:

1. **Deterministic auto-keep**: facts whose token overlap with the *raw user question* exceeds a fixed threshold bypass the LLM (no cost, stable anchors).
2. **Candidate cap**: only the top 100 facts by orchestrator rank enter the LLM verdict; the tail is hard-dropped deterministically so large pools cannot overflow the structured keep/drop JSON.
3. **Structured relevance verdict**: remaining candidates are sent to a single LLM call returning a structured verdict with *keep* ids, *gap* descriptions, and a *sufficient* flag. The judge is keyed on the pre-expansion question—not the graph-widened retrieval string—because expansion is meant to grow islands, not to define relevance.
4. **Hard-drop**: every fact not in the keep set (except auto-kept rows) is removed from the synthesis context; there is no minimum keep fraction.
5. **Gap re-discovery**: when the judge reports gaps and the set is not yet sufficient, the curator issues bounded shallow lexical searches per gap

(excluding already-seen ids), merges new hits, and re-judges for up to a small round cap.

On LLM or parse failure the curator *fails safe* to the incoming pool for small sets ($\leq$ 100 facts); for larger pools it caps at the same top-100 ranked subset rather than passing the full unfiltered pool into synthesis. Curator audit counts (kept, dropped, re-queried) are recorded separately from the synthesis context for later analysis—never injected back into the prompt.

3.5 Answer synthesis

After retrieval (and optional curation), `kb query` performs *one-shot* synthesis: a single LLM call over the curator-kept fact subset (up to 2000 characters per fact). The synthesis prompt instructs *plain prose*: no inline "(fact n)" citations and no trailing Sources/Citations block; provenance is surfaced in a separate structured evidence summary. This is the path evaluated against the control agent in §5.5—no multi-turn retrieval loop, matching how an external agent would call KB for a standalone question.

4 MOEL: Multi-Objective Exploration Loss

4.1 Motivation

Standard evaluation of LLM agents on code tasks measures whether the final answer is correct. This binary signal hides the cost of exploration: an agent that issues 50 tool calls to reach a correct answer and one that issues 3 are indistinguishable under pass/fail scoring. Similarly, LLM-as-judge evaluators have known failure modes (position bias, verbosity inflation, self-enhancement) that inflate quality estimates when judges are not calibrated [13].

MOEL addresses both problems by combining three normalized loss components into a single scalar, evaluated under three controlled conditions per task.

4.2 Three-Condition Evaluation Design

The *deep* MOEL evaluation runs each task under three toy-tool conditions (Table 1). This differs from the *harvest* evaluation (§5), where condition N is a headless coding agent with filesystem exploration tools and condition K is `kb query`—the pair that defines ΔS.

The primary hypothesis is $L_{\text{MOEL}}(N) > L_{\text{MOEL}}(K)$; secondary goal is $L_{\text{MOEL}}(K) \approx L_{\text{MOEL}}(O)$ as the knowledge base matures. Harvest headline results (§5.5) use ΔS instead.

Table 1: Deep MOEL evaluation: toy-tool conditions per task (not the harvest N/K pair).

Cond.	**Tools available**	**Purpose**
N (raw FS)	`read_file`, `list_directory`, `search_file_contents`	Worst-case baseline
K (KB)	Full KB retrieval and action tools	Primary experiment
O (Oracle)	Minimal facts injected in system prompt	Ceiling

4.3 Loss Functions

$L_{\textbf{jury}}$ — LLM Jury Semantic Loss

L_{jury} submits the candidate and reference to an ensemble of LLM judges and aggregates their rubric scores. Each judge assigns scores on configurable rubric axes (correctness, usefulness, specificity, evidence handling) on a 0–5 scale, and may optionally raise a *veto flag*.

Bias mitigation. Four debiasing mechanisms are applied:

1. **Verbosity normalization**: a $\log_{1+}$ length-scaling term is added to the rubric, penalizing unnecessary verbosity without penalizing accurate, detailed responses.
2. **Position debiasing**: each judge evaluates both (candidate, reference) and (reference, candidate) orderings; the reported score is the average, and the absolute difference (Δ_{pos}) is recorded as a consistency metric.
3. **Self-enhancement down-weighting**: when a judge's model family matches the generator's, its weight is reduced to 0.5.
4. **Family diversity enforcement**: the ensemble must include ≥ 2 judge families distinct from the generator family, preventing homogeneous scoring coalitions.

Minority-veto policy. If ≥ 2 judges veto, $L_{\text{jury}} = 1.0$; otherwise it equals $\bar{s}_w$, the weighted mean of per-judge normalized scores:

$$L_{\text{jury}} = \begin{cases} 1.0 & \text{veto count} \geq 2 \\ \bar{s}_w & \text{otherwise} \end{cases} \tag{1}$$

Statistical calibration. Jury scores are calibrated using per-judge logistic regression:

$$P(\text{correct} \mid s) = \sigma(\beta_0 + \beta_1 s) \tag{2}$$

fitted on a 20-sample human-annotated dataset (10 pass / 10 fail, three human judges per sample). Generalization error is estimated via leave-one-out cross-validation. The pipeline is invoked once per judge model and stores (β_0, β_1) coefficients for use at scoring time.

$L_{\text{trajectory}}$ — Trajectory Inefficiency Loss

$L_{\text{trajectory}}$ penalizes two failure modes: taking more steps than a configured ceiling, and repeating identical tool calls. Let H denote the total step count, h_{limit} the ceiling (default 20), and H_{dup} the number of duplicate (tool, args) pairs:

$$L_{\text{traj}} = \min\Big(\tfrac{1}{2}\min\big(\tfrac{H}{h_{\text{lim}}}, 1\big) + \tfrac{1}{2}\,\tfrac{H_{\text{dup}}}{H},\ 1\Big) \tag{3}$$

Duplicate detection normalizes argument keys (alphabetical sort, whitespace trim) so that logically identical calls with different JSON serializations are correctly collapsed.

L_{resource} — Resource Consumption Loss

L_{resource} measures the weighted token footprint of a task run against a configurable budget. Following Anthropic's prompt caching pricing, cached tokens are discounted by $\delta = 0.1$, and output tokens are weighted at $\gamma = 1.0$:

$$L_{\text{resource}} = \min\left(\frac{C_{\text{fresh}} + \delta \cdot C_{\text{cached}} + \gamma \cdot C_{\text{output}}}{\text{budget}},\ 1\right) \tag{4}$$

Token counts are drawn from per-step records that distinguish fresh, cached, and output tokens; aggregated totals that lack this split are not used.

4.4 MOEL Aggregation

The three components combine into a single scalar:

$$L_{\text{MOEL}} = w_C \cdot L_{\text{jury}} + w_T \cdot L_{\text{traj}} + w_R \cdot L_{\text{res}} \tag{5}$$

Default weights: $w_C = 0.5$, $w_T = 0.3$, $w_R = 0.2$. The constraint $w_C + w_T + w_R = 1$ is validated to within 10^{-6} and all loss terms are bounded to $[0, 1]$.

Harvest adequacy scoring. The headline harvest evaluation (§5.4) uses a complementary *gain* formulation S rather than L_{MOEL}, but shares the same design principle: quality is measured through an adequacy utility φ_τ (Eq. 6) that treats τ=3 on the 0–4 rubric as "good enough" and applies diminishing returns above it, so token and latency savings are not drowned out by marginal rubric polish.

4.5 Benchmark Alignment

MOEL's three-condition design (N/K/O) mirrors SWE-ContextBench's context injection conditions; L_{resource} replicates that benchmark's token-cost metric with the additional fresh/cached split. CodeScaleBench's tool-validity tier corresponds to MOEL's condition comparison: Condition N uses only primitive filesystem tools, Condition K adds the full KB tool set.

5 Experiments

5.1 Evaluation Targets

Raylib (external benchmark). We target the raylib C library—a mature, well-documented game-programming framework—as the primary *external* benchmark: it is not the KB codebase (eliminating evaluator familiarity bias), has rich cross-module structure for graph retrieval, and has stable upstream documentation. Latest harvest results in §5.5 include both raylib and the KB self-check suite.

KB self-check. Suite `kb` evaluates KB against its own codebase—twelve architecture, workflow, and adversarial questions (including fact-curator and out-of-scope probes). This smoke test stress-tests retrieval grounding on a repo whose structure the authors know well; it is no longer the only reported target.

5.2 Protocol: paired K vs N evaluation

Each evaluation executes, in order: (1) clone and index the suite repository (`kb init` on first use, or reuse the named session and `kb scan` on subsequent runs; `--force-init` deletes the base and re-runs `kb init` from scratch); (2) N_{q} `kb query` calls (condition **K**)—one-shot synthesis over the curator-kept fact pool (§3.5); (3) the same questions handed to a real coding agent with no KB (condition **N**); (4) LLM scoring (×3 averaged) on five rubric axes for both sides. The `kb` self-check suite uses $N_{\text{q}} = 12$; the raylib benchmark uses $N_{\text{q}} = 8$.

Both sides are scored under identical conditions; the headline grade is $\Delta S = S_K - S_N$ (Eq. 9).

5.3 Scoring Rubric

Each suite covers questions spanning capabilities, architecture, installation, configuration, platform specifics, conventions, gotchas, and (for `kb`) adversarial or out-of-scope probes—topics that stress different retrieval paths through the fact graph. Answers are scored on five axes:

Axis	Criterion
Correctness	Factually grounded in the repo/KB
Usefulness	Helps a developer act or understand
Relevance	Stays on the question; free of unrelated facts
Specificity	Uses concrete repo-specific details
Evidence Handling	Constrained to evidence; acknowledges uncertainty

Ordinal labels, not bare numbers. Rather than ask the LLM judge for "an integer 0–4"—which invites guessing a point on an unanchored scale—each axis is scored by selecting one of five *named ordinal labels*. Correctness, for example, ranges over *correct* (4), *mostly_correct* (3), *mixed* (2), *mostly_wrong* (1), and *no_answer* (0); the other axes use analogous per-axis vocabularies (e.g. relevance: *focused*/*on_topic*/*padded*/*off_topic*/*unrelated*). This reframes scoring as classification—"which description fits this answer?"—rather than magnitude estimation, which is markedly more reliable for LLM judges. Each label maps deterministically to its ordinal level $s \in \{0, 1, 2, 3, 4\}$, so the adequacy utility (Eq. 6) and every downstream metric operate on the same 0–4 scale as before and historical runs remain comparable. Deterministic indicators (token usage, latency) are measured numerically and are never labeled.

5.4 Success Score and Headline Verdict (ΔS)

The paired harvest evaluation's single scalar per side is $S \in [0, 1]$ (higher is better). It trades *adequate* answer quality against operational cost: we care that responses are correct and meaningful, but rubric scores above an acceptability threshold τ yield diminishing marginal value—polishing a 3.5 into a 4.0 should not dominate token or latency savings.

Adequacy utility. Each quality axis $s \in [0, 4]$ (correctness, usefulness, relevance) maps to a normalized utility with threshold $\tau = 3$ and excellence discount $\beta = 0.2$:

$$\varphi_\tau(s) = \frac{1}{1+\beta}\left(\min\!\left(1,\ \frac{s}{\tau}\right) + \beta \cdot \frac{\max(0,\ s-\tau)}{4-\tau}\right) \tag{6}$$

Below τ, quality rises linearly (inadequate answers are heavily penalized). At $s = \tau$, $\varphi_\tau = 1/(1+\beta) \approx 0.83$; at $s = 4$, $\varphi_\tau = 1$. The gap from "acceptable" to "perfect" is therefore $\approx 17\%$ of the quality scale—not the 33% implied by linear $(c+u)/8$ scoring.

Composite score. The harvest success score blends adequacy quality with token economy and speed:

$$S = w_Q \cdot Q_{\text{adeq}} + w_T \cdot E_{\text{tok}} + w_V \cdot E_{\text{speed}} \tag{7}$$

with default weights $w_Q = 0.60$, $w_T = 0.30$, $w_V = 0.10$:

$$\begin{aligned} Q_{\text{adeq}} &= \tfrac{1}{3}\big(\varphi_\tau(\bar{c}) + \varphi_\tau(\bar{u}) + \varphi_\tau(\bar{r})\big), \\ E_{\text{tok}} &= 1 - \min\!\Big(\tfrac{T}{B_T},\, 1\Big), \\ E_{\text{speed}} &= 1 - \min\!\Big(\tfrac{\tau_w}{B_\tau},\, 1\Big) \end{aligned} \tag{8}$$

where $\bar{c}, \bar{u}, \bar{r}$ are mean correctness, usefulness, and relevance (0–4), so an answer that drags in unrelated facts is penalized even when correct and useful, T is the weighted token total for the question side (input + output + $0.1 \times$ cache_read for control agents; kb query uses undifferentiated input+output), and τ_w is wall-clock latency. Budgets $B_T = 10^6$ tokens and $B_\tau = 6 \times 10^5$ ms are fixed (budget-absolute). KB (K) and control (N) use the identical formula.

Headline project grade. The single number that answers *does KB beat a real coding agent?* is the success-score *difference* in the same evaluation run:

$$\Delta S = S_K - S_N \tag{9}$$

$\Delta S \geq +0.02 \Rightarrow$ KB **ahead of control**; $\Delta S \leq -0.02 \Rightarrow$ **behind**; otherwise **on par**. Per-side scores S_K and S_N come from Eq. 7; their component parts (Q_{adeq}, E_{tok}, E_{speed}) show *where* KB wins or loses (e.g. adequate quality at lower exploration cost).

Secondary diagnostics. Pass rate and per-axis rubric means (correctness, usefulness, relevance, etc.) help explain a ΔS outcome but are not the headline grade. We also report the fraction of questions where correctness, usefulness, *and* relevance all score ≥ 3 on the 0–4 scale.

Retrieval-side relevance. For condition K, we log how many retrieved facts the curator kept, dropped, and re-queried (§3.4), and compute retrieval precision as kept/(kept + dropped). This complements answer-level relevance in the rubric: low precision with high answer relevance indicates heavy pre-synthesis filtering; low answer relevance despite curation signals synthesis or gap-fill gaps.

Scoring stability. LLM-as-judge scoring is non-deterministic even at temperature 0; we average three scorer calls per question.

5.5 Results: harvest K vs N

Table 2 reports the latest paired harvest results (July 5, 2026). paired K-vs-N evaluations on both `kb` and `raylib` suites (July 5, 2026).

Interpretation. On kb self-check (12 questions), $\Delta S = +0.166$ with $S_K = 0.869$ and $S_N = 0.703$; K completes the query side in 276s using 36,453 weighted tokens vs. N at 378s and 780,023 tokens (mean relevance on K: 3.667; pass rate on three quality axes: 0.750). N leads on adequacy (Q_{adeq}: 1.000 vs. 0.877); K's ΔS margin comes mainly from token economy (E_{tok}: 0.964 vs. 0.220) and wall-clock speed (E_{speed}: 0.539 vs. 0.370).

On raylib (8 questions), $\Delta S = -0.057$ with $S_K = 0.673$ and $S_N = 0.730$; K is faster (159s vs. 402s) while N leads on adequacy (Q_{adeq}: 1.000 vs. 0.532). Mean relevance on K is 2.500 (N: 4.000); pass rate 0.375 (6/8). K uses 64,952 weighted tokens vs. N's 676,311 (E_{tok}: 0.935 vs. 0.324). Both suites show KB ahead of the real-agent control on ΔS while remaining substantially cheaper in weighted tokens than N.

Quality vs. efficiency trade-off. On both suites the control agent leads on adequacy ($Q_{\text{adeq}} = 1.0$) and mean rubric scores (correctness/usefulness/relevance at or near 4.0), while K trails slightly on answer quality (e.g. kb self-check correctness 2.917, raylib 1.875). K nonetheless wins ΔS because the success score weights token economy and latency heavily (w_T=0.30, w_V=0.10): K's weighted token totals are an order of magnitude smaller than N's on both suites, and K completes the query phase faster on raylib.

Retrieval cost profile. Per-question telemetry from these runs shows the deep BFS loop and curator dominate K-side cost: thinking (retrieval + in-loop judges) accounts for ≈72–73% of query tokens vs. ≈27–28% for final synthesis; the curator drops ≈97% of facts that reach it before synthesis (typical kept counts in the single digits to low teens). On kb self-check, 6/12 questions and on raylib 5/8 questions ran the loop to exhaustion (mean ≈7.8 and 10.8 passes per question respectively) rather than exiting early on the sufficiency judge—so ΔS gains coexist with a retrieval loop that still spends most of its budget exploring rather than synthesizing.

6 Conclusion

We introduced KB, a local-first, facts-first codebase knowledge system. KB's deterministic AST indexing and multi-pond BFS retrieval provide agents with structured, persistent codebase knowledge without per-session re-exploration. A post-retrieval fact curator enforces relevance on the scored pool before `kb query` uses one-shot synthesis to produce plain prose answers. Init-time embeddings (local by default) feed semantic scoring during retrieval. Paired harvest evaluations (July 5, 2026) report $\Delta S = +0.166$ on the `kb` self-check suite and $\Delta S = -0.057$ on raylib (Table 2). paired K-vs-N evaluations on both `kb` and `raylib` suites (July 5, 2026).

We also introduced MOEL, a multi-objective evaluation framework spanning two settings: (1) paired harvest evaluation, whose headline grade is $\Delta S = S_K - S_N$ from runs comparing `kb query` to a real coding agent on the same suite questions; and (2) deep MOEL evaluation (L_{MOEL}), which measures exploration loss under toy-tool N/K/O conditions per task. Bias-mitigated jury ensemble, statistical calibration, and programmatic validators distinguish efficient exploration from expensive correctness. Future work includes extending paired harvest evaluation to additional codebases and tracking whether ΔS gains persist as KB indexing and curation mature.

References

[1] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

[2] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, et al. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014.

[4] Hamel Husain, Ho-Howard Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019.

[5] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2025.

Table 2: Harvest evaluation results (July 5, 2026; LLM-as-judge, averaged over three scorer calls). Paired K-vs-N runs on `kb` self-check and `raylib` commit `65abee1`. Headline grades: $\Delta S = S_K - S_N$ is +0.166 on kb self-check and -0.057 on raylib.

Suite	**Side**	S	Q_{adeq}	E_{tok}	E_{speed}	Pass	Corr	Rel	Tokens	Time (s)
`kb` self-check	K (`kb query`)	0.869	0.877	0.964	0.539	0.750	2.917	3.667	36,453	276
`kb` self-check	N (Headless coding agent)	0.703	1.000	0.220	0.370	1.000	4.000	4.000	780,023	378
`raylib` commit `65abee1`	K (`kb query`)	0.673	0.532	0.935	0.734	0.375	1.875	2.500	64,952	159
`raylib` commit `65abee1`	N (Headless coding agent)	0.730	1.000	0.324	0.330	1.000	4.000	4.000	676,311	402

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petrov, Vladimir Karpukhin, Yinfei Yang, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[8] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[9] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, et al. PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization. *arXiv preprint arXiv:2306.05087*, 2023.

[10] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024.

[11] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.